

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**  
**TRƯỜNG KINH TẾ PHENIKAA**  
**KHOA DU LỊCH – KHÁCH SẠN**

---



**BÀI TẬP LỚN**

**ĐỀ TÀI:**

**PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG ĐẶT LỊCH**  
**HƯỚNG DẪN VIÊN DU LỊCH DỰA TRÊN KIẾN TRÚC MVC**

**Giảng viên: Th.S. Phạm Đức Bắc**

**Nhóm sinh viên: Hà Thị Ánh Ngọc - 23016275**

**Nguyễn Hùng Mạnh - 23015477**

**Lê Mai Anh - 23015455**

**Nhóm: 10**

**Lớp: Lập trình ứng dụng web trong du lịch (N01)**

**HÀ NỘI, 06/2026**

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**  
**TRƯỜNG KINH TẾ PHENIKAA**  
**KHOA DU LỊCH – KHÁCH SẠN**

---

**ĐỀ TÀI:**  
**PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG ĐẶT LỊCH**  
**HƯỚNG DẪN VIÊN DU LỊCH DỰA TRÊN KIẾN TRÚC MVC**

**Nhóm sinh viên: Hà Thị Ánh Ngọc - 23016275**

**Nguyễn Hùng Mạnh - 23015477**

**Lê Mai Anh - 23015455**

**Hà Nội – 06/2026**



## Contents

|                                                                                                                                                                                                                                                                                                                                    |          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>MỞ ĐẦU .....</b>                                                                                                                                                                                                                                                                                                                | <b>1</b> |
| 1.1.1 Phát biểu bài toán.....                                                                                                                                                                                                                                                                                                      | 3        |
| 1.2 Mục tiêu và phạm vi thiết kế .....                                                                                                                                                                                                                                                                                             | 5        |
| 1.2.1 Mục tiêu hệ thống.....                                                                                                                                                                                                                                                                                                       | 5        |
| 1.3. Cơ sở lý thuyết về Kiến trúc và Bảo mật .....                                                                                                                                                                                                                                                                                 | 6        |
| 1.3.1. Kiến trúc Model-View-Controller (MVC).....                                                                                                                                                                                                                                                                                  | 6        |
| 1.4. Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn .....                                                                                                                                                                                                                                                                      | 8        |
| 1.4.1. Framework PHP Laravel và Trình đóng gói Eloquent ORM .....                                                                                                                                                                                                                                                                  | 8        |
| 2.1 Biểu đồ Use case và Đặc tả Tác nhân .....                                                                                                                                                                                                                                                                                      | 11       |
| 2.1.1 Xác định và phân loại Tác nhân (Actors).....                                                                                                                                                                                                                                                                                 | 11       |
| 2.1.2. Thiết kế sơ đồ Use Case tổng thể.....                                                                                                                                                                                                                                                                                       | 12       |
| 2.1.3. Đặc tả chi tiết các Use Case trọng tâm.....                                                                                                                                                                                                                                                                                 | 12       |
| 2.2. Phân tích và Thiết kế Phân hệ Chức năng.....                                                                                                                                                                                                                                                                                  | 15       |
| 2.2.2. Phân hệ Khách hàng (User Web Interface) .....                                                                                                                                                                                                                                                                               | 15       |
| 2.1.3. Phân hệ Quản trị viên (Admin Command Control) .....                                                                                                                                                                                                                                                                         | 16       |
| 2.2 Thiết kế Cơ sở dữ liệu Logic (Database Schema) .....                                                                                                                                                                                                                                                                           | 16       |
| 2.2.1 Định nghĩa cấu trúc chi tiết các bảng dữ liệu .....                                                                                                                                                                                                                                                                          | 16       |
| 2.2.2 Mô hình Quan hệ Thực thể liên kết (Entity-Relationship Diagram) .....                                                                                                                                                                                                                                                        | 19       |
| 2.3 Sơ đồ Luồng Xử lý Nghiệp vụ Hệ thống (Request Lifecycle) .....                                                                                                                                                                                                                                                                 | 19       |
| 3.1.1. Tổ chức mã nguồn theo Nguyên lý Đơn trách nhiệm (SRP).....                                                                                                                                                                                                                                                                  | 21       |
| 3.1.2. Định tuyến RESTful và Tối ưu hóa Bảng điều hướng.....                                                                                                                                                                                                                                                                       | 21       |
| 3.2. Xây dựng Lớp Phòng thủ Bảo mật (Defense in Depth) .....                                                                                                                                                                                                                                                                       | 22       |
| 3.2.1 Ngăn chặn Leo thang Đặc quyền (Privilege Escalation Prevention) .....                                                                                                                                                                                                                                                        | 22       |
| 3.2.2. Bảo vệ tính Toàn vẹn Dữ liệu Đầu vào (Mass Assignment & Validation) .....                                                                                                                                                                                                                                                   | 23       |
| Một trong những lỗ hổng nguy hiểm nhất trong quá trình khởi tạo dữ liệu là Mass Assignment Vulnerability (Gán dữ liệu hàng loạt). Khi khách hàng đăng ký tài khoản mới, biểu mẫu HTML chỉ gửi lên full_name, email, và password. Tuy nhiên, kẻ tấn công có thể cố tình thêm trường "role": "admin" vào nội dung gói tin POST. .... | 23       |
| 3.3. Kiến trúc Truy vấn Dữ liệu và Tối ưu hóa (Data Access Layer) .....                                                                                                                                                                                                                                                            | 23       |
| 3.3.1. Triệt tiêu Nút thắt cổ chai N+1 Query .....                                                                                                                                                                                                                                                                                 | 23       |
| 3.4. Thiết kế Thuật toán Điều phối và Chống Trùng Lịch (Anti-Overlapping Algorithm).....                                                                                                                                                                                                                                           | 24       |
| 3.4.1 Đặc tả Thuật toán Logic Toán học.....                                                                                                                                                                                                                                                                                        | 24       |
| 3.4.2. Hiện thực hóa Thuật toán qua Query Builder .....                                                                                                                                                                                                                                                                            | 25       |
| 3.5 Cấu trúc Tầng Giao diện và Tương tác Thời gian thực .....                                                                                                                                                                                                                                                                      | 26       |
| 3.5.1 Bảo vệ chống Tấn công XSS (Cross-Site Scripting) .....                                                                                                                                                                                                                                                                       | 26       |

|                                                                                                                                                                                              |           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 3.5.2 Kỹ thuật DOM Manipulation ước lượng chi phí .....                                                                                                                                      | 26        |
| <b>4.1 Môi trường triển khai thực nghiệm .....</b>                                                                                                                                           | <b>28</b> |
| Hệ thống được triển khai và cấu hình cục bộ (Localhost) để phục vụ cho quá trình kiểm thử hộp đen (Black-box Testing) và kiểm thử hộp trắng (White-box Testing) của nhóm phát triển. ....    | 28        |
| 4.1.1 Cấu hình phần cứng (Hardware Specification) .....                                                                                                                                      | 28        |
| 4.1.2 Môi trường phần mềm và Công nghệ (Software Environment) .....                                                                                                                          | 28        |
| <b>4.2 Kịch bản kiểm thử hệ thống (System Test Cases) .....</b>                                                                                                                              | <b>29</b> |
| Để đảm bảo hệ thống vận hành trơn tru và không xảy ra các lỗi hỏng logic nghiệp vụ, nhóm phát triển đã tiến hành xây dựng các kịch bản kiểm thử bám sát vào các luồng Use Case cốt lõi. .... | 29        |
| 4.2.1 Kiểm thử Module Bảo mật và Phân quyền (RBAC) .....                                                                                                                                     | 29        |
| 4.2.2 Kiểm thử Thuật toán Chống trùng lặp (Anti-Overlapping) .....                                                                                                                           | 30        |
| <b>4.3. Giao diện trực quan và Kết quả đạt được .....</b>                                                                                                                                    | <b>30</b> |
| 4.3.1 Giao diện Dành cho Khách hàng (Customer UI) .....                                                                                                                                      | 31        |
| 4.3.2 Giao diện Trung tâm Điều hành (Admin Dashboard) .....                                                                                                                                  | 32        |
| <b>4.4 Đánh giá ưu điểm và hạn chế .....</b>                                                                                                                                                 | <b>33</b> |
| 4.4.1 Đánh giá ưu điểm nổi bật .....                                                                                                                                                         | 33        |
| 4.4.2 Những mặt Hạn chế còn tồn tại .....                                                                                                                                                    | 33        |
| <b>KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN .....</b>                                                                                                                                                    | <b>34</b> |
| <b>TÀI LIỆU THAM KHẢO .....</b>                                                                                                                                                              | <b>35</b> |

## MỞ ĐẦU

Sự phát triển mạnh mẽ của ngành du lịch đã kéo theo nhu cầu tìm kiếm các dịch vụ hỗ trợ lẫn hành chất lượng cao, đặc biệt là dịch vụ hướng dẫn viên du lịch. Tuy nhiên, qua khảo sát thực tế, việc kết nối giữa du khách và hướng dẫn viên hiện nay phần lớn vẫn diễn ra thủ công thông qua các hội nhóm mạng xã hội hoặc các công ty môi giới trung gian. Quá trình này dẫn đến nhiều hạn chế như: khó kiểm chứng năng lực của hướng dẫn viên, thông tin lịch trình dễ bị trùng lặp, và đặc biệt là du khách khó khăn trong việc tìm kiếm người phù hợp với ngôn ngữ và địa điểm mong muốn.

Nhận thấy những hạn chế đó, cùng với mong muốn áp dụng các kiến thức đã học về kỹ nghệ phần mềm vào thực tiễn, nhóm chúng em quyết định chọn đề tài: **"Phân tích, thiết kế và xây dựng hệ thống đặt lịch hướng dẫn viên du lịch dựa trên kiến trúc MVC"**.

### Mục tiêu của đề tài:

- **Về mặt lý thuyết:** Nghiên cứu và ứng dụng framework Laravel (PHP), kiến trúc MVC, và các công nghệ giao diện hiện đại (Blade Template, Bootstrap) vào việc phát triển ứng dụng Web.
- **Về mặt thực tiễn:** Xây dựng thành công một hệ thống phần mềm hoàn chỉnh cho phép du khách dễ dàng tìm kiếm, lọc theo ngôn ngữ/địa điểm và đặt lịch hướng dẫn viên trực tuyến; đồng thời cung cấp cho Quản trị viên (Admin) một không gian điều hành tập trung để quản lý nhân sự, kiểm duyệt đơn đặt lịch và thống kê doanh thu theo thời gian thực.

**Bố cục của báo cáo:** Ngoài phần Mở đầu và Kết luận, báo cáo được chia thành 4 chương chính:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng:** Trình bày bối cảnh bài toán và các nền tảng công nghệ được sử dụng.
- **Chương 2: Phân tích và thiết kế hệ thống:** Phân rã các chức năng qua biểu đồ Use Case, sơ đồ cơ sở dữ liệu (ERD) và luồng xử lý nghiệp vụ.
- **Chương 3: Thiết kế chi tiết và Hiện thực hóa mã nguồn:** Đi sâu vào kiến trúc thư mục, cách cấu hình Route, Controller, Model và kỹ thuật xử lý nghiệp vụ.

- **Chương 4: Kết quả thực nghiệm và Đánh giá:** Trình bày các giao diện đã hoàn thiện và kịch bản kiểm thử hệ thống.

# CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

## 1.1 Bài toán thực tế và Nhu cầu hệ thống

### 1.1.1 Phát biểu bài toán

Trong bối cảnh nền kinh tế toàn cầu hóa và xu hướng số hóa mạnh mẽ hiện nay, ngành công nghiệp không khói (du lịch) đang chứng kiến những sự chuyển dịch cơ cấu vô cùng lớn. Đặc biệt, xu hướng du lịch tự túc đang dần thay thế các mô hình tour du lịch trọn gói truyền thống. Du khách ngày nay mong muốn có những trải nghiệm mang tính cá nhân hóa cao, tự do khám phá văn hóa bản địa thông qua lăng kính của những người hướng dẫn viên địa phương. Điều này đặt ra một thách thức lớn đối với việc tổ chức, điều phối và kết nối giữa cung - Hướng dẫn viên và cầu - Khách du lịch.

Tuy nhiên, qua khảo sát thực tế tại thị trường du lịch hiện tại, quy trình kết nối và quản lý dịch vụ hướng dẫn viên tự do phần lớn vẫn mang tính chất thủ công, phân tán và thiếu tính chuyên nghiệp. Các hướng dẫn viên thường tự tìm kiếm khách hàng thông qua các hội nhóm mạng xã hội hoặc phụ thuộc vào các đại lý môi giới trung gian với mức chiết khấu cao. Cách tiếp cận này bộc lộ những lỗ hổng và điểm nghẽn nghiêm trọng trong quy trình vận hành dịch vụ:

- **Sự phân tán và thiếu minh bạch về thông tin (Information Asymmetry):** Khách du lịch rất khó để đánh giá chính xác năng lực, kinh nghiệm, độ thông thạo ngôn ngữ và tính chuyên nghiệp của hướng dẫn viên trước khi quyết định thuê. Dữ liệu về hướng dẫn viên nằm rải rác ở nhiều nền tảng khác nhau mà không có một bộ lọc chuẩn hóa nào.
- **Rủi ro xung đột lịch trình (Double Booking):** Việc thiếu hụt một hệ thống kiểm soát trạng thái theo thời gian thực dẫn đến hiện tượng điều phối trùng lặp. Một hướng dẫn viên có thể vô tình nhận hai đoàn khách có khung thời gian chồng chéo nhau do ghi chép sổ sách sai sót. Ngược lại, có những khoảng thời gian hướng dẫn viên hoàn toàn trống lịch nhưng khách hàng lại không có cách nào để biết và tiếp cận.
- **Khó khăn trong quản lý nghiệp vụ đối với các công ty lữ hành:** Đối với các đơn vị cung cấp dịch vụ, việc quản lý hàng chục, hàng trăm hướng dẫn viên cộng tác viên là một bài toán khó. Việc tổng hợp số lượng ngày công, trạng thái lịch đặt, và đặc biệt là kiểm soát doanh thu thủ công tốn rất nhiều thời gian, nhân lực và dễ xảy ra sai sót dữ liệu.

Từ những bất cập thực tế trên, việc nghiên cứu, phân tích và xây dựng một nền tảng phần mềm tập trung hóa, tự động hóa quy trình đặt lịch hướng dẫn viên du lịch là một yêu cầu vô cùng cấp thiết nhằm nâng cao năng lực chuyển đổi số trong ngành du lịch.

### 1.1.2. Nhu cầu giải quyết phần mềm

Để giải quyết triệt để các bài toán cốt lõi đã phát biểu, hệ thống phần mềm được thiết kế và xây dựng bắt buộc phải đáp ứng toàn diện ba trục nhu cầu kỹ thuật cốt lõi sau đây:

1. **Trục Quản lý Tài nguyên và Ràng buộc Dữ liệu Bền vững (Relational Data Integrity):** Hệ thống cần tổ chức và chuẩn hóa cơ sở dữ liệu quan hệ đạt các tiêu chuẩn chuẩn hóa (Normal Forms). Việc thiết lập cấu trúc bảng phải phản ánh chính xác các ràng buộc vật lý trong thực tế. Mỗi hướng dẫn viên phải được gán với nhiều ngôn ngữ và nhiều địa bàn hoạt động khác nhau (Mỗi quan hệ Nhiều - Nhiều), đồng thời trạng thái công việc của họ (Sẵn sàng, Đang bận) phải được quản lý chặt chẽ.
2. **Trục Tự động hóa Nghiệp vụ và Đồng bộ hóa Trạng thái (Automated Business Logic):** Hệ thống phải cung cấp một cơ chế tính toán tự động các tham số kinh tế. Khi khách hàng lựa chọn khoảng thời gian thuê (Ngày bắt đầu và Ngày kết thúc), phần mềm sẽ tự động tính toán số ngày thực tế và nhân với đơn giá thuê theo ngày của hướng dẫn viên để đưa ra tổng chi phí (Total Price) ngay tại tầng logic của ứng dụng trước khi ghi nhận vào cơ sở dữ liệu. Thuật toán phải có khả năng kiểm tra chéo (Cross-check) để chặn đứng mọi yêu cầu đặt lịch nếu khoảng thời gian đó hướng dẫn viên đã có lịch bận.
3. **Trục Bảo mật Hệ thống và Phân quyền Nghiêm ngặt (Access Control Framework):** Phần mềm cần thiết lập một rào cản bảo mật vững chắc thông qua các bộ lọc yêu cầu HTTP (Middleware). Khách hàng chỉ được quyền tra cứu, đặt lịch và quản lý đơn của chính mình. Toàn bộ các quyền hạn mang tính chất thay đổi cấu trúc dữ liệu dùng chung (Thêm, sửa, xóa hướng dẫn viên, ngôn ngữ, địa điểm) hoặc tác động đến vòng đời của đơn đặt lịch (Phê duyệt, Hoàn thành) phải được cô lập hoàn toàn cho tài khoản Quản trị viên (Admin), chặn đứng nguy cơ khai thác lỗ hổng leo thang đặc quyền từ phía Client.

## 1.2 Mục tiêu và phạm vi thiết kế

### 1.2.1 Mục tiêu hệ thống

Mục tiêu tổng quát của đề tài nghiên cứu này là ứng dụng quy trình kỹ nghệ phần mềm (Software Engineering) hiện đại để phân tích, thiết kế và hiện thực hóa một ứng dụng Web đặt lịch hướng dẫn viên du lịch (Tourist Guide Booking) toàn diện. Hệ thống hướng tới việc đạt được các mục tiêu kỹ thuật cụ thể sau đây:

- **Về mặt kiến trúc và mã nguồn:** Triển khai thành công mẫu thiết kế Model-View-Controller (MVC) chuẩn công nghiệp thông qua khung làm việc Laravel. Đảm bảo cấu trúc mã nguồn tường minh, sạch sẽ, tách biệt hoàn toàn giữa logic tương tác cơ sở dữ liệu, logic điều hướng nghiệp vụ và tầng hiển thị giao diện để nâng cao khả năng bảo trì và mở rộng hệ thống trong tương lai.
- **Về mặt quản trị nghiệp vụ:** Cung cấp cho ban điều hành (Admin) một không gian làm việc tập trung (Admin Panel) trực quan. Tự động hóa 100% việc thu thập và tính toán dữ liệu thống kê, hiển thị tổng số lượng đơn đặt lịch, số lượng đơn đang chờ duyệt, và tổng doanh thu thực tế để hỗ trợ ra quyết định kinh doanh.
- **Về mặt trải nghiệm người dùng (UX/UI):** Xây dựng giao diện hướng người dùng (User-centric Design), giúp du khách có thể dễ dàng sử dụng các bộ lọc (Filters) đa luồng để tìm kiếm hướng dẫn viên theo khoảng giá, theo số năm kinh nghiệm, hoặc theo đích danh ngôn ngữ, địa điểm một cách nhanh chóng nhất. Tích hợp các cơ chế cảnh báo lỗi nhập liệu trực tiếp tại Client để mang lại trải nghiệm mượt mà nhất.

### 1.2.2. Phạm vi thiết kế

Để đảm bảo tính khả thi, độ tập trung chuyên sâu và hoàn thiện sản phẩm phần mềm chất lượng cao trong khuôn khổ của đề tài, dự án xác định rõ phạm vi thiết kế và triển khai giới hạn như sau:

- **Về hạ tầng kiến trúc:** Dự án tập trung phát triển ứng dụng Web dựa trên mô hình kết xuất phía máy chủ (Server-Side Rendering) truyền thống kết hợp với các công cụ tối ưu hóa Frontend. Dự án không triển khai các giải pháp kiến trúc hướng dịch vụ (Microservices) nhằm giữ cho hệ thống tinh gọn, tập trung tối đa vào xử lý nghiệp vụ lõi.
- **Về phân hệ người dùng:** Hệ thống tập trung giải quyết triệt để quy trình đặt lịch của Khách hàng (Customer) thông qua luồng đăng ký/đăng nhập chuẩn và phân hệ điều hành tối cao của Quản trị viên (Admin). Hệ thống hiện chưa tích hợp không gian làm việc riêng rẽ cho từng Hướng dẫn viên (Guide Portal), việc điều phối trạng thái của hướng dẫn viên hiện tại sẽ do Admin đảm nhiệm.
- **Về giải pháp thanh toán:** Hệ thống tập trung xử lý luồng logic nghiệp vụ của đơn đặt lịch và tự động hóa tính toán tổng số tiền giao dịch. Việc thanh toán thực tế được giả định thực hiện qua phương thức thỏa thuận (tiền mặt hoặc chuyển khoản bên ngoài hệ thống) khi đơn hàng được Admin chuyển sang trạng thái "Đã xác nhận"(Confirmed). Hệ thống chưa kết nối trực tiếp với các cổng thanh toán trung gian trực tuyến của bên thứ ba (như VNPay, MoMo, hay ZaloPay).

### 1.3. Cơ sở lý thuyết về Kiến trúc và Bảo mật

#### 1.3.1. Kiến trúc Model-View-Controller (MVC)

Mẫu kiến trúc Model-View-Controller (MVC) là một trong những nền tảng kinh điển và phổ biến nhất trong công nghệ phần mềm hiện đại, đóng vai trò là kiến trúc cốt lõi xuyên suốt quá trình xây dựng hệ thống của đề án này. Triết lý cốt lõi của MVC là nguyên lý tách biệt các mối quan tâm, chia một ứng dụng phần mềm thành ba thành phần biệt lập có chức năng chuyên biệt:

1. **Model (Thành phần Mô hình Dữ liệu):** Đảm nhận vai trò quản lý toàn bộ trạng thái dữ liệu và logic nghiệp vụ cốt lõi của ứng dụng. Model trực tiếp giao tiếp với hệ quản trị cơ sở dữ liệu quan hệ, thực thi các truy vấn dữ liệu, kiểm tra các ràng buộc thực thể và thực hiện các tính toán logic. Model hoàn toàn độc lập với giao diện hiển thị. Trong hệ thống, các lớp như User, Booking, Guide, Location chính là

các Model đại diện cho các bảng dữ liệu vật lý dưới CSDL.

2. **View (Thành phần Hiển thị Giao diện):** Chịu trách nhiệm biên dịch dữ liệu nhận được từ Model thành cấu trúc giao diện trực quan hiển thị cho người dùng cuối (thường là HTML, CSS, và JavaScript). View không chứa các logic xử lý nghiệp vụ phức tạp mà chỉ chứa các cú pháp lặp, rẽ nhánh cơ bản để render dữ liệu động vào mã nguồn tĩnh.
3. **Controller (Thành phần Bộ điều khiển Logic):** Đóng vai trò là cầu nối điều phối, điều khiển luồng tương tác giữa người dùng, Model và View. Khi người dùng gửi một yêu cầu (HTTP Request) từ trình duyệt, Controller sẽ tiếp nhận, phân tích tham số, gọi các phương thức xử lý tương ứng trong Model để thay đổi dữ liệu, sau đó đóng gói dữ liệu kết quả và đẩy sang tầng View để kết xuất giao diện trả về cho Client (HTTP Response).

Sự cô lập và phân tầng nghiêm ngặt này mang lại những ưu thế công nghệ vượt trội: Giúp mã nguồn có tính module hóa cao, dễ dàng thực hiện kiểm thử; khi thay đổi cấu trúc giao diện ở View, logic nghiệp vụ ở Controller và Model hoàn toàn được giữ nguyên; đồng thời giảm thiểu tối đa xung đột mã nguồn khi triển khai phát triển dự án theo nhóm.

### 1.3.2. Mô hình kiểm soát truy cập (RBAC)

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) là mô hình bảo mật chuẩn công nghiệp được ứng dụng để giải quyết bài toán phân quyền và quản lý tài nguyên trong hệ thống. Nguyên lý cốt lõi của RBAC là quyền truy cập vào các tài nguyên phần mềm không được gán trực tiếp cho từng tài khoản người dùng cụ thể, mà được gán gián tiếp thông qua các vai trò trung gian được định nghĩa trước trong hệ thống.

Trong kiến trúc cơ sở dữ liệu của dự án, thuộc tính role được triển khai dưới dạng kiểu dữ liệu liệt kê (Enum) trong bảng users, giới hạn nghiêm ngặt với hai giá trị thực thể: 'customer' và 'admin'. Logic vận hành của mô hình RBAC trong hệ thống được hiện thực hóa qua quy trình xử lý luồng nghiêm ngặt sau:

- **Định danh và Duy trì Phiên làm việc (Authentication & Session):** Khi người

dùng gửi thông tin đăng nhập hợp lệ, hệ thống sẽ khởi tạo một phiên làm việc (Session ID) được lưu trữ an toàn phía Server và đính kèm Cookie định danh tương ứng ở trình duyệt Client. Phiên làm việc này bọc toàn bộ thông tin thực thể của người dùng hiện tại bao gồm khóa chính id, email, và giá trị role.

- **Kiểm soát và Đánh chặn tại Tầng Lọc (Middleware Isolation):** Đối với mọi HTTP Request hướng tới các tài nguyên nhạy cảm (như các đường dẫn quản trị bắt đầu bằng /admin/), hệ thống áp dụng cơ chế đánh chặn chủ động thông qua lớp Middleware bảo mật có tên EnsureUserIsAdmin. Middleware này sẽ trích xuất thông tin người dùng từ Session, thực hiện đối sánh điều kiện logic. Nếu vai trò không phải là Admin, hệ thống lập tức hủy luồng yêu cầu và trả về mã trạng thái lỗi HTTP 403 (Forbidden).

Giải pháp này loại bỏ hoàn toàn các rủi ro bảo mật liên quan đến việc rò rỉ dữ liệu hoặc lỗi logic phân quyền, đảm bảo tính toàn vẹn hệ thống ở mức tối đa.

## 1.4. Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn

### 1.4.1. Framework PHP Laravel và Trình đóng gói Eloquent ORM

Hệ thống quản lý dịch vụ đặt lịch hướng dẫn viên được xây dựng trên nền tảng Backend mạnh mẽ là Framework PHP Laravel (Phiên bản 10.x), kết hợp với môi trường PHP 8.1 trở lên. Laravel được lựa chọn nhờ vào hệ sinh thái công cụ toàn diện và cơ chế bảo mật mặc định (Secure by Default) vô cùng nghiêm ngặt, giúp lập trình viên tập trung tối đa vào thiết kế logic nghiệp vụ thay vì phải xử lý các vấn đề hạ tầng thô sơ.

- **Cơ chế bảo vệ an toàn dữ liệu tự động:** Laravel tích hợp sẵn các màng lọc an ninh chống lại ba loại lỗ hổng bảo mật phổ biến nhất trên môi trường ứng dụng web. Lỗ hổng giả mạo yêu cầu chéo trang (CSRF) được chặn đứng thông qua việc ép buộc đính kèm một mã thông báo độc quyền (CSRF Token) vào mọi Request dạng POST/PUT/DELETE. Lỗ hổng chèn mã lệnh script độc hại (XSS) được vô hiệu hóa hoàn toàn nhờ cơ chế tự động escape dữ liệu của Blade Engine. Lỗ hổng chèn mã độc cơ sở dữ liệu (SQL Injection) được loại bỏ tận gốc nhờ kỹ thuật liên kết tham số (Parameter Binding) ẩn dưới tầng Database của Laravel.

- **Bản chất xử lý ánh xạ đối tượng Eloquent ORM:** Điểm nhấn công nghệ cốt lõi

của Laravel là trình đóng gói Eloquent ORM (Object-Relational Mapping). Eloquent trừu tượng hóa các bảng dữ liệu vật lý trong MySQL thành các lớp lập trình hướng đối tượng độc lập. Thay vì phải viết các câu lệnh SQL thuần phức tạp với các phép nối JOIN công kênh, Eloquent cho phép định nghĩa các mối quan hệ thực thể ngay trong mã nguồn PHP thông qua các hàm liên kết như `hasMany()` và `belongsToMany()`. Dự án đã ứng dụng triệt để tính năng này để xây dựng bảng trung gian (`guide_languages`, `guide_locations`) giúp kết nối nhiều ngôn ngữ, nhiều địa điểm vào một người hướng dẫn viên một cách linh hoạt nhất.

#### 1.4.2. Kiến trúc Frontend với Blade Template, Bootstrap và Vite

Để tối ưu hóa giao diện hiển thị, đảm bảo tính thích ứng (Responsive UI) trên mọi thiết bị và mang lại trải nghiệm người dùng hiện đại, hệ thống ứng dụng mô hình phát triển Frontend đồng bộ từ Server kết hợp với công cụ đóng gói siêu tốc Vite.

- **Blade Template Engine:** Là công cụ kết xuất giao diện mặc định của Laravel, cho phép sử dụng mã PHP trực tiếp trong các file HTML một cách an toàn và tinh gọn. Bằng việc tận dụng cơ chế kế thừa layout (`@extends`) và chia nhỏ các thành phần giao diện (Components/Partials như các khối `alert.blade.php` hoặc `_form.blade.php`), dự án đã thực hiện rất tốt nguyên tắc lập trình DRY (Don't Repeat Yourself), giúp giảm thiểu mã nguồn lặp lại và dễ dàng nâng cấp giao diện toàn hệ thống chỉ với một vài chỉnh sửa nhỏ.
- **Tích hợp Bootstrap Framework và Vanilla JS:** Khác với việc phải cài đặt các hệ sinh thái Single Page Application (SPA) công kênh, dự án tận dụng Bootstrap 5 để thiết kế hệ thống lưới (Grid) và các linh kiện UI sắc nét. Đồng thời, đối với các tương tác động phản hồi ngay tại Client (như việc tự động tính toán tổng số tiền khi người dùng thay đổi ngày bắt đầu và kết thúc đặt lịch), dự án sử dụng trực tiếp ngôn ngữ Vanilla JavaScript được nhúng an toàn qua directive `@push('scripts')`, giúp hệ thống luôn đạt hiệu năng cao nhất mà không bị phụ thuộc vào quá nhiều thư viện bên thứ ba.
- **Trình đóng gói tài nguyên Vite:** Hệ thống sử dụng Vite (được cấu hình qua `vite.config.js` và `package.json`) làm công cụ Module Bundler thay thế cho Laravel Mix trước đây. Việc này mang lại tốc độ biên dịch (Hot Module Replacement) gần như tức thì trong quá trình phát triển, đồng thời tối ưu hóa, nén (Minify) các tài

nguyên CSS/JS thành những tệp có dung lượng siêu nhỏ khi triển khai lên môi trường thực tế (Production).

## CHƯƠNG 2: PHÂN TÍCH HỆ THỐNG

### 2.1 Biểu đồ Use case và Đặc tả Tác nhân

Trong quy trình phát triển và kỹ nghệ phần mềm, việc xác định rõ ràng ranh giới hệ thống, các tác nhân tương tác và các ca sử dụng (Use case) là bước đi tiên quyết giúp định hình toàn bộ kiến trúc logic của ứng dụng. Hệ thống Quản lý Đặt lịch Hướng dẫn viên (Tourist Guide Booking) được xây dựng dựa trên sự phân tách nghiêm ngặt quyền hạn truy cập của các thực thể sử dụng.

Dựa trên cấu trúc phân phối đường dẫn hệ thống (Routing Framework Architecture), mô hình nghiệp vụ được phân hóa thành ba nhóm tác nhân chính tương tác trực tiếp với các tài nguyên phần mềm thông qua các giao thức mạng bảo mật: Khách vắng lai (Guest), Khách hàng đã xác thực (Customer) và Quản trị viên hệ thống (System Administrator).

#### 2.1.1 Xác định và phân loại Tác nhân (Actors)

- **Khách vắng lai (Guest):** Là những người dùng chưa thực hiện định danh hoặc phiên làm việc (Session) chưa được thiết lập trên hệ thống. Nhóm tác nhân này chỉ có quyền tiếp cận các tài nguyên tĩnh hoặc động mang tính chất công khai, không nhạy cảm như tra cứu danh sách hướng dẫn viên, xem chi tiết hồ sơ hướng dẫn viên, thực hiện đăng ký tài khoản mới hoặc đăng nhập vào hệ thống.
- **Khách hàng đã xác thực (Customer):** Là tác nhân người dùng phổ thông đã vượt qua màn lọc xác thực (auth middleware) và được cấp phát một Session ID hợp lệ. Ngoài việc kế thừa toàn bộ quyền hạn của Khách vắng lai, nhóm người dùng này được phép tương tác với cơ sở dữ liệu để khởi tạo đơn đặt lịch (Booking Requests), theo dõi lịch trình cá nhân và kiểm tra lịch sử trạng thái đơn hàng của chính mình.
- **Quản trị viên (Admin):** Là tác nhân tối cao sở hữu thuộc tính quyền hạn đặc biệt (role = 'admin') trong bảng dữ liệu thực thể users. Tác nhân này được bảo vệ nghiêm ngặt bởi màn lọc kiểm soát truy cập chuyên biệt (EnsureUserIsAdmin), có toàn quyền can thiệp vào vòng đời của dữ liệu hệ thống (CRUD) bao gồm hướng dẫn viên, ngôn ngữ, địa điểm, quản lý khách hàng và nắm giữ quyền sinh sát trong

việc kiểm duyệt, phê duyệt hoặc hoàn thành/hủy bỏ đơn đặt lịch.

### 2.1.2. Thiết kế sơ đồ Use Case tổng thể

Sơ đồ Use Case tổng thể bao quát các cụm chức năng chính: Khách hàng thực hiện luồng Đăng ký/Đăng nhập, Tìm kiếm & Lọc Hướng dẫn viên, Đặt lịch và Quản lý Lịch sử Đặt; Admin thực hiện Quản lý Danh mục (Địa điểm, Ngôn ngữ), Quản lý Hướng dẫn viên, Quản lý Khách hàng, Xử lý Đơn đặt lịch và Xem Báo cáo thống kê.

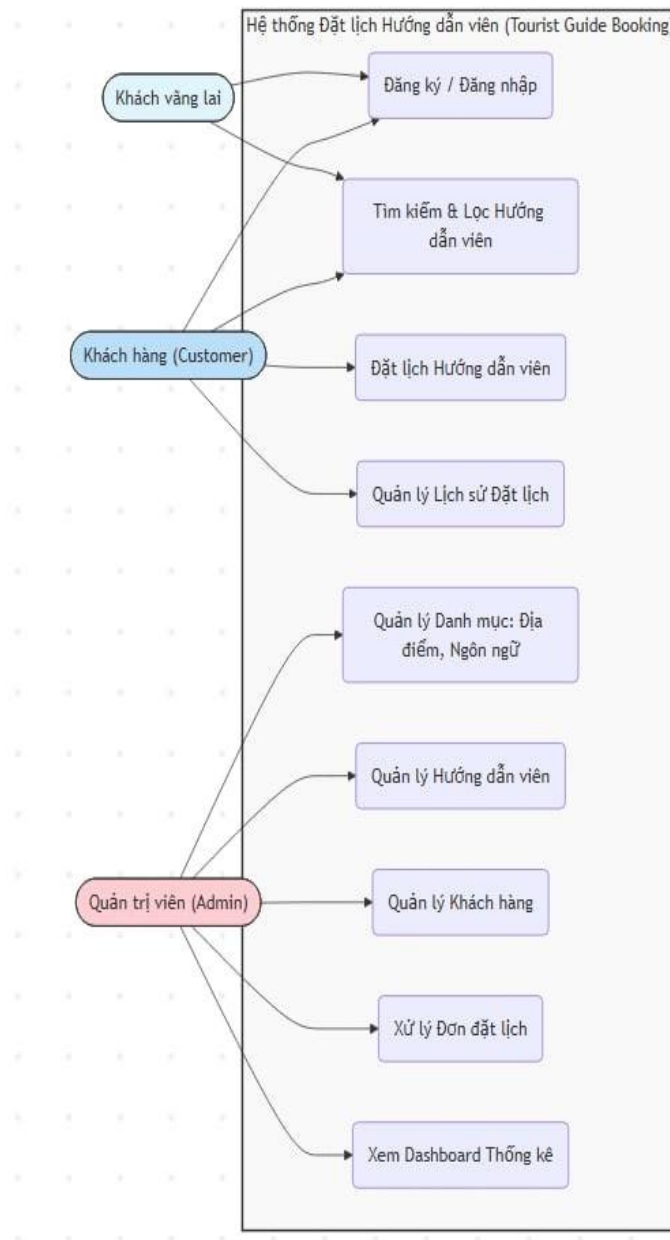
### 2.1.3. Đặc tả chi tiết các Use Case trọng tâm

Để hiện thực hóa mã nguồn một cách chính xác tại tầng xử lý logic, các ca sử dụng cốt lõi của hệ thống cần được đặc tả cấu trúc luồng sự kiện một cách tường minh:

#### 1. Ca sử dụng: Tra cứu và Lọc Hướng dẫn viên (Search/Filter Guides)

- **Tác nhân kích hoạt:** Guest, Customer.
- **Tiền điều kiện:** Hệ thống phải có các hồ sơ hướng dẫn viên được cấu hình trạng thái available hoặc busy.
- **Luồng sự kiện chính:** Người dùng nhập từ khóa hoặc chọn bộ lọc (Ngôn ngữ, Địa điểm, Khoảng giá, Số năm kinh nghiệm, Giới tính) tại giao diện. Hệ thống tiếp nhận HTTP Request gửi kèm tham số truy vấn, sử dụng cơ chế Query Builder để truy xuất dữ liệu từ bảng guides, thực hiện phân trang (12 bản ghi/trang) và trả về kết quả hiển thị cho người dùng.

Hình 2.1: Biểu đồ Use Case tổng thể phân rã các phân hệ chức năng



## 2. Ca sử dụng: Đặt lịch Hướng dẫn viên (Create Booking)

- **Tác nhân kích hoạt:** Customer.
- **Tiền điều kiện:** Người dùng đã đăng nhập thành công và hướng dẫn viên lựa chọn phải có trạng thái hoạt động available.
- **Luồng sự kiện chính:** Người dùng điền biểu mẫu bao gồm ngày bắt đầu, ngày kết thúc và ghi chú. Hệ thống tiến hành xác thực dữ liệu đầu vào thông qua lớp kiểm thử (StoreBookingRequest). Thuật toán kiểm tra chéo sẽ quét cơ sở dữ liệu để đảm bảo hướng dẫn viên không bị trùng lịch đã được xác nhận hoặc đang chờ duyệt trong khoảng thời gian này. Nếu dữ liệu hợp lệ, tầng Logic tính toán tổng số ngày nhân với đơn giá, ghi dữ liệu xuống bảng bookings với trạng thái mặc định ban đầu là pending và điều hướng về trang lịch sử.

## 3. Ca sử dụng: Điều phối và Phê duyệt đơn đặt lịch (Approve/Cancel Booking)

- **Tác nhân kích hoạt:** Admin.
- **Tiền điều kiện:** Tài khoản điều hành phải vượt qua bộ lọc quyền hạn cấp cao; đơn hàng cần xử lý phải ở trạng thái hợp lệ để chuyển đổi.
- **Luồng sự kiện chính:** Admin truy cập màn hình quản trị đơn đặt lịch. Khi thao tác xác nhận (confirm) hoặc hủy bỏ (cancel), hệ thống kiểm tra logic bất biến của trạng thái. Nếu đơn hàng đang ở pending, nó sẽ được cập nhật thành confirmed. Nếu đơn đã confirmed, nó có thể chuyển sang completed. Mọi sự thay đổi đều cập nhật bền vững xuống CSDL.

## 2.2. Phân tích và Thiết kế Phân hệ Chức năng

Hệ thống được module hóa thành ba phân hệ chức năng độc lập về mặt logic nhưng có sự liên kết chặt chẽ về mặt chia sẻ tài nguyên dữ liệu thông qua kiến trúc máy chủ tập trung.

### 2.2.1. Phân hệ Xác thực và Định danh (Authentication Module)

Phân hệ này chịu trách nhiệm thiết lập hàng rào an ninh đầu tiên cho ứng dụng Web, quản lý toàn bộ vòng đời phiên làm việc của tài khoản dựa trên cấu trúc lõi bảo mật mã nguồn.

- **Đăng ký tài khoản (User Registration):** Tiếp nhận các thuộc tính full\_name, email, password, phone từ Client. Hệ thống áp dụng thuật toán băm bảo mật một chiều (Hash::make) để chuyển đổi mật khẩu thuần sang chuỗi ký tự mã hóa an toàn trước khi lưu vào CSDL.
- **Đăng nhập và Tạo phiên (User Login & Session Management):** Đối sánh thông tin tài khoản thu thập từ biểu mẫu với dữ liệu lưu trữ qua Auth::attempt(). Nếu trùng khớp, cơ chế session()->regenerate() của máy chủ lập tức được kích hoạt nhằm làm mới Session ID của phiên làm việc, ngăn chặn triệt để lỗ hổng tấn công cố định phiên (Session Fixation Attack).

### 2.2.2. Phân hệ Khách hàng (User Web Interface)

Phân hệ này cung cấp không gian trải nghiệm tối giản, trực quan hóa luồng dữ liệu nghiệp vụ cho khách hàng có nhu cầu tìm kiếm dịch vụ lữ hành.

- **Trang chủ và Tra cứu Hướng dẫn viên:** Hiện thị danh sách phân trang chứa hình ảnh avatar, ngôn ngữ, địa điểm, số năm kinh nghiệm và đơn giá. Tích hợp bộ công cụ lọc tìm kiếm động bằng Eloquent Query linh hoạt.
- **Trang Quản lý lịch sử cá nhân:** Bảng dữ liệu phân trang hiển thị toàn bộ lịch trình người dùng đã thực hiện đặt lịch trong quá khứ và hiện tại (user()->bookings()). Trạng thái đơn hàng được trực quan hóa bằng các nhãn màu (Badge Component) sắc nét để người dùng tiện theo dõi tiến độ phê duyệt.

### 2.1.3. Phân hệ Quản trị viên (Admin Command Control)

Là trung tâm đầu não điều khiển luồng hoạt động kinh doanh tổng thể của toàn ứng dụng.

- **Bảng điều khiển Thống kê tổng hợp (Dashboard Statistics):** Sử dụng lớp `StatisticsService` để tính toán các thông số kinh tế cốt lõi: Tổng số lượt đặt lịch, số lượng đơn đang chờ duyệt, tổng doanh thu tài chính từ các đơn hoàn thành.
- **Quản lý danh mục Tài nguyên (CRUD Resources Management):** Cung cấp đầy đủ các giao diện biểu mẫu Thêm/Sửa/Xóa đối với các thực thể nền tảng: Hướng dẫn viên (guides), Khách hàng (users), Ngôn ngữ (languages) và Địa điểm (locations). Phân hệ tích hợp luồng xử lý tệp tin (File Upload) bảo mật lưu trữ cục bộ đối với ảnh đại diện của Hướng dẫn viên.

## 2.2 Thiết kế Cơ sở dữ liệu Logic (Database Schema)

Để đảm bảo hệ thống phần mềm vận hành với hiệu năng cao và duy trì tính toàn vẹn tham chiếu, cơ sở dữ liệu quan hệ (RDBMS) của hệ thống Tourist Guide Booking được thiết kế chuẩn hóa. Toàn bộ lược đồ thực thể và mối quan hệ ràng buộc vật lý được khai báo thông qua cấu trúc mã nguồn di trú dữ liệu (Database Migrations).

### 2.2.1 Định nghĩa cấu trúc chi tiết các bảng dữ liệu

#### 1. Bảng users (Quản lý tài khoản người dùng và Admin)

Lưu trữ thông tin định danh và phân quyền đặc quyền của hệ thống.

| Tên trường | Kiểu dữ liệu | Ràng buộc          | Mô tả nghiệp vụ                |
|------------|--------------|--------------------|--------------------------------|
| id         | BigInt       | Primary Key, Auto  | Khóa chính định danh           |
| full_name  | Varchar(100) | Not Null           | Họ và tên chủ tài khoản        |
| email      | Varchar(100) | Unique, Not Null   | Địa chỉ Email (Dùng đăng nhập) |
| password   | Varchar(255) | Not Null           | Chuỗi mật khẩu đã băm          |
| phone      | Varchar(20)  | Nullable           | Số điện thoại liên hệ          |
| address    | Varchar(255) | Nullable           | Địa chỉ thường trú             |
| role       | Enum         | Default 'customer' | Vai trò: admin hoặc            |

|            |           |          |                    |
|------------|-----------|----------|--------------------|
|            |           |          | customer           |
| created_at | Timestamp | Not Null | Thời gian khởi tạo |

Bảng 2.1: Cấu trúc chi tiết bảng dữ liệu users

2. **Bảng guides (Quản lý hồ sơ hướng dẫn viên)** Lưu trữ thông tin cá nhân và nghiệp vụ của đội ngũ hướng dẫn viên.

3. **Các bảng languages, locations và Bảng trung gian (Pivot)** Lưu trữ các danh mục phụ trợ và giải quyết mối quan hệ Nhiều - Nhiều (N-N) thông qua các bảng trung gian guide\_languages và guide\_locations.

4. **Bảng bookings (Quản lý giao dịch đặt lịch)** Lưu trữ toàn bộ thông tin đơn hàng do người dùng khởi tạo, tham chiếu chéo giữa khách hàng (users) và hướng dẫn viên (guides).

| Tên trường       | Kiểu dữ liệu  | Ràng buộc           | Mô tả nghiệp vụ                       |
|------------------|---------------|---------------------|---------------------------------------|
| id               | BigInt        | Primary Key, Auto   | Khóa chính định danh                  |
| full_name        | Varchar(100)  | Not Null            | Họ và tên HDV                         |
| gender           | Enum          | male, female, other | Giới tính                             |
| phone / email    | Varchar       | Unique (email)      | Thông tin liên lạc                    |
| experience_years | Integer       | Default 0           | Số năm kinh nghiệm làm nghề           |
| price_per_day    | Decimal(10,2) | Not Null            | Đơn giá thuê theo ngày                |
| avatar           | Varchar(255)  | Nullable            | Đường dẫn file ảnh đại diện           |
| status           | Enum          | Default 'available' | Trạng thái: available, busy, inactive |

Bảng 2.2: Cấu trúc chi tiết bảng dữ liệu guides

| Tên Bảng        | Khóa ngoại 1 | Khóa ngoại 2 | Mục đích                  |
|-----------------|--------------|--------------|---------------------------|
| guide_languages | guide_id     | language_id  | Ràng buộc HDV và Ngôn ngữ |

|                 |          |             |                                     |
|-----------------|----------|-------------|-------------------------------------|
|                 |          |             | thông thạo                          |
| guide_locations | guide_id | location_id | Ràng buộc HDV và Địa điểm hoạt động |

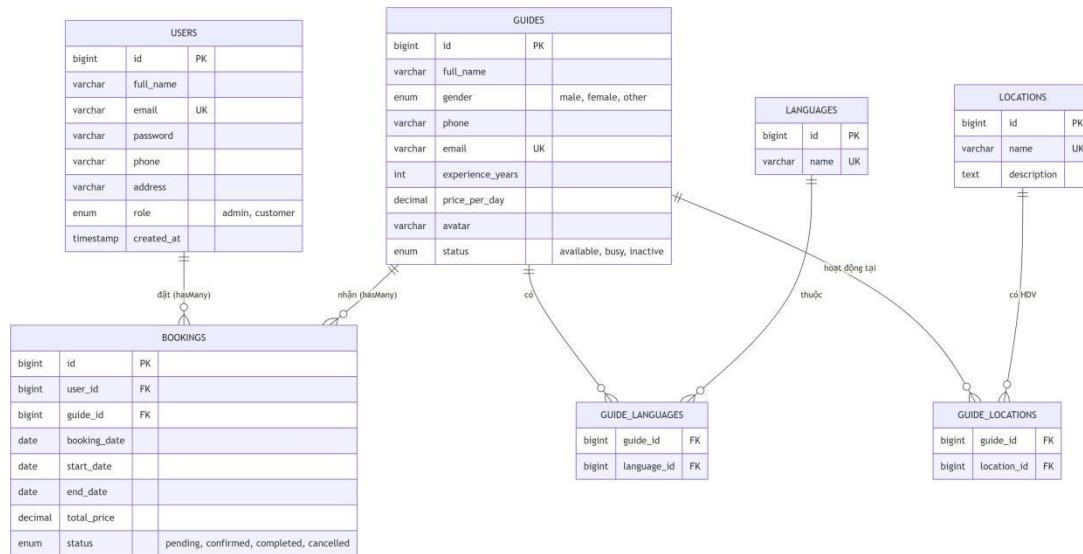
Bảng 2.3: Cấu trúc các bảng quan hệ trung gian

| Tên trường   | Kiểu dữ liệu  | Ràng buộc         | Mô tả nghiệp vụ                                      |
|--------------|---------------|-------------------|------------------------------------------------------|
| id           | BigInt        | Primary Key, Auto | Khóa chính mã đơn đặt lịch                           |
| user_id      | BigInt        | Foreign Key       | Mã khách hàng đặt lịch                               |
| guide_id     | BigInt        | Foreign Key       | Mã hướng dẫn viên được đặt                           |
| booking_date | Date          | Not Null          | Ngày thực hiện tạo đơn                               |
| start_date   | Date          | Not Null          | Ngày bắt đầu lịch trình                              |
| end_date     | Date          | Not Null          | Ngày kết thúc lịch trình                             |
| total_price  | Decimal(10,2) | Nullable          | Tổng tiền (Đã nhân số ngày)                          |
| status       | Enum          | Default 'pending' | Trạng thái: pending, confirmed, cancelled, completed |

Bảng 2.4: Cấu trúc chi tiết bảng dữ liệu bookings.

### 2.2.2 Mô hình Quan hệ Thực thể liên kết (Entity-Relationship Diagram)

Hình 2.2: Sơ đồ mối quan hệ giữa các thực thể vật lý dưới cơ sở dữ liệu (ERD)



Lược đồ ERD phản ánh cấu trúc logic :

- **Mối quan hệ users - bookings (1 - N):** Một khách hàng có thể đặt nhiều lịch (hasMany), nhưng một đơn đặt lịch chỉ thuộc về một khách hàng duy nhất (belongsTo). Các ràng buộc Cascade được áp dụng để xóa đơn khi người dùng bị xóa.
- **Mối quan hệ guides - bookings (1 - N):** Một hướng dẫn viên có thể nhận nhiều lượt đặt lịch khác nhau qua thời gian, nhưng mỗi đơn đặt lịch bắt buộc phải ánh xạ tới một hướng dẫn viên cụ thể.
- **Mối quan hệ guides - languages/locations (N - N):** Một hướng dẫn viên có thể thông thạo nhiều ngôn ngữ và nhận dẫn tour ở nhiều địa điểm, ngược lại một địa điểm có thể có nhiều hướng dẫn viên hoạt động. Quan hệ này được giải quyết trọn vẹn thông qua các bảng Pivot với phương thức belongsToMany.

### 2.3 Sơ đồ Luồng Xử lý Nghiệp vụ Hệ thống (Request Lifecycle)

Để hiểu rõ cách thức mã nguồn ứng dụng Laravel xử lý dữ liệu và điều hướng luồng thông tin khi nhận được yêu cầu từ trình duyệt Client, chúng ta tiến hành phân tích vòng đời của một yêu cầu tạo đơn đặt lịch (Booking Request):

1. **Giai đoạn Gửi Yêu cầu & Định tuyến (Routing & Validation):** Khách hàng điền form chọn start\_date, end\_date và bấm "Đặt lịch" tại giao diện show.blade.php. Một POST

Request được gửi tới `/guides/{guide}/book`. Tuyến đường này được bảo vệ bởi auth middleware, chuyển tiếp luồng dữ liệu vào lớp `StoreBookingRequest`.

2. **Giai đoạn Xác thực và Chống trùng lịch (Business Logic Check):** Tại Form Request, hệ thống không chỉ kiểm tra định dạng ngày mà còn kích hoạt hàm `withValidator`. Hàm này truy vấn CSDL để tìm xem có đơn bookings nào của `guide_id` này đang ở trạng thái `confirmed` hoặc `pending` mà bị giao lấp thời gian (`overlapping`) với ngày khách chọn không. Nếu có trùng lặp, hệ thống sẽ gây luồng và ném lỗi trả về trình duyệt.
3. **Giai đoạn Xử lý Controller (Controller Phase):** Nếu vượt qua kiểm thử, `BookingController` sẽ tiếp nhận dữ liệu. Controller sử dụng đối tượng Carbon để tính số ngày chênh lệch (`diffInDays`). Sau đó, hệ thống tự động nhân số ngày này với `price_per_day` của Model Guide để ra biến số `total_price` an toàn.
4. **Giai đoạn Ghi CSDL và Kết xuất (Database & Response):** Hàm `Booking::create()` được gọi để lưu bản ghi vật lý xuống MySQL. Cuối cùng, Controller thực hiện `redirect()->route('bookings.index')` kèm theo thông báo flash "Đặt lịch thành công", giao diện tiếp nhận và render một khối Alert thông báo màu xanh cho người dùng.

### CHƯƠNG 3. THIẾT KẾ CHI TIẾT VÀ HIỆN THỰC HÓA MÃ NGUỒN

Sau khi hoàn tất quá trình phân tích và thiết kế hệ thống ở mức logic, bước tiếp theo và cũng là khâu mang tính quyết định trong quy trình kỹ nghệ phần mềm là hiện thực hóa các bản thiết kế thành mã nguồn có thể thực thi. Trong các hệ thống phần mềm quy mô lớn, việc chỉ đơn thuần viết mã để chức năng "có thể chạy được" là chưa đủ; mã nguồn còn phải đáp ứng các tiêu chuẩn khắt khe về tính bảo trì (`Maintainability`), hiệu năng truy vấn (`Performance`) và đặc biệt là an toàn thông tin (`Application Security`). Chương này sẽ trình bày một cách tường minh và sâu sắc cách thức nhóm phát triển ứng dụng các nguyên lý lập trình hướng đối tượng (`OOP`), các mẫu thiết kế (`Design Patterns`) và các cơ chế phòng thủ sâu của framework Laravel để xây dựng nên hệ thống Đặt lịch Hướng dẫn viên.

#### 3.1 Kiến trúc Thư mục và Cơ chế Định tuyến (Routing Architecture)

### 3.1.1. Tổ chức mã nguồn theo Nguyên lý Đơn trách nhiệm (SRP)

Hệ thống được tổ chức tuân thủ tuyệt đối nguyên lý Đơn trách nhiệm (Single Responsibility Principle) nằm trong bộ quy tắc SOLID. Việc phân rã thư mục không chỉ là sự sắp xếp vật lý mà còn là sự phân tách về mặt logic nghiệp vụ (Separation of Concerns). Toàn bộ mã nguồn cốt lõi được đặt trong không gian tên (Namespace) App, phân bổ thành các tầng chuyên biệt:

- **Tầng Models:** Không chỉ đóng vai trò ánh xạ cơ sở dữ liệu (ORM), mà còn là "Rich Domain Model" đóng gói các hằng số trạng thái, logic tính toán số ngày và định dạng chuỗi, ngăn không cho các logic này rò rỉ sang Controller.
- **Tầng Http/Controllers:** Đóng vai trò là các bộ điều phối luồng (Traffic Controllers). Controller trong hệ thống được thiết kế theo hướng "Skinny" (mỏng), chỉ chịu trách nhiệm nhận dữ liệu đã được làm sạch, gọi Model xử lý và trả về View.
- **Tầng Http/Requests:** Được thiết kế như một "Bức tường lửa" (Firewall) nội bộ, chuyên trách việc kiểm tra tính hợp lệ của dữ liệu (Validation) trước khi dữ liệu chạm đến Controller.

### 3.1.2. Định tuyến RESTful và Tối ưu hóa Bảng điều hướng

Thay vì định nghĩa từng đường dẫn GET, POST, PUT, DELETE một cách thủ công dẫn đến bảng định tuyến bị phình to và khó kiểm soát, hệ thống ứng dụng triệt để cơ chế `Route::resource` của Laravel. Cơ chế này tự động sinh ra các điểm cuối (Endpoints) tuân thủ hoàn toàn theo tiêu chuẩn thiết kế RESTful API.

```
Route::middleware(['auth', 'admin'])
->prefix('admin')
->name('admin.')
->group(function () {
    Route::resource('guides', AdminGuideController::class)->except(['show']);
    Route::resource('languages', LanguageController::class)->except(['show']);
    // Các routes nghiệp vụ khác...
});
```

Điểm đáng chú ý trong tư duy thiết kế ở đây là việc sử dụng phương thức `except(['show'])`. Đối với các danh mục thực thể đơn giản như Ngôn ngữ (Languages) hoặc Địa điểm (Locations), việc hiển thị một trang chi tiết riêng biệt là dư thừa về mặt nghiệp vụ. Do đó, việc chủ động loại bỏ hàm `show` ngay từ tầng định tuyến giúp bảng Route trở nên tinh gọn, đồng thời giảm thiểu các bề mặt tấn công tiềm năng (Attack Surface) đối với các điểm cuối không được sử dụng.

### 3.2. Xây dựng Lớp Phòng thủ Bảo mật (Defense in Depth)

Bảo mật ứng dụng web là một quá trình phòng thủ nhiều lớp. Hệ thống đặt lịch không chỉ tin tưởng vào các cơ chế bảo mật giao diện (Frontend) mà còn thiết lập các rào cản nghiêm ngặt ngay từ tầng xử lý HTTP Request của máy chủ.

#### 3.2.1 Ngăn chặn Leo thang Đặc quyền (Privilege Escalation Prevention)

Trong hệ thống, sự phân định ranh giới giữa tài khoản Khách hàng (Customer) và Quản trị viên (Admin) là bài toán sống còn. Nếu chỉ ản các nút bấm chuyển trang trên giao diện HTML, những kẻ tấn công (Attackers) có am hiểu kỹ thuật hoàn toàn có thể sử dụng các công cụ chặn bắt gói tin (như Burp Suite hay Postman) để gửi trực tiếp các truy vấn HTTP POST/PUT vào các đường dẫn quản trị.

Nhằm triệt tiêu hoàn toàn rủi ro này, hệ thống hiện thực hóa một lớp Middleware đánh chặn có tên `EnsureUserIsAdmin`. Middleware này hoạt động như một người gác cổng (Gatekeeper) thực thụ.

```
public function handle(Request $request, Closure $next): Response
{
    if (!$request->user()->isAdmin()) {
        abort(403, __('auth.unauthorized'));
    }

    return $next($request);
}
```

Logic vận hành: Trước khi bắt kỳ một Request nào tiến vào thư mục `AdminController`, nó bắt buộc phải đi xuyên qua hàm `handle()` này. Hệ thống sẽ trích xuất phiên làm việc (Session) hiện tại, truy xuất thực thể `User` và gọi phương thức

isAdmin(). Nếu vai trò (Role) không khớp, quá trình xử lý bị cắt đứt ngay lập tức ở cấp độ Kernel, trả về mã trạng thái HTTP 403 Forbidden. Điều này tiết kiệm tài nguyên tính toán cho máy chủ và đảm bảo tính cô lập dữ liệu tuyệt đối.

### 3.2.2. Bảo vệ tính Toàn vẹn Dữ liệu Đầu vào (Mass Assignment & Validation)

Một trong những lỗ hổng nguy hiểm nhất trong quá trình khởi tạo dữ liệu là Mass Assignment Vulnerability (Gán dữ liệu hàng loạt). Khi khách hàng đăng ký tài khoản mới, biểu mẫu HTML chỉ gửi lên full\_name, email, và password. Tuy nhiên, kẻ tấn công có thể cố tình thêm trường "role": "admin" vào nội dung gói tin POST.

Để vô hiệu hóa kỹ thuật tấn công này, toàn bộ các Model trong hệ thống (User, Guide, Booking) đều được thiết lập mảng bảo vệ \$fillable.

```
Protected $fillable = [  
    'full_name',  
    'email',  
    'password',  
    'phone',  
    'address',  
    'role',  
];
```

Mặc dù role nằm trong \$fillable, nhưng tại RegisterController, mã nguồn sử dụng cơ chế gán cứng (Hardcoded) thay vì lấy trực tiếp từ \$request->all(): 'role' => User::ROLE\_CUSTOMER. Kết hợp với cơ chế liên kết tham số (Parameter Binding) của PDO tích hợp sẵn trong Laravel Eloquent, hệ thống miễn nhiễm hoàn toàn với mọi hình thức tấn công chèn mã độc cơ sở dữ liệu (SQL Injection).

## 3.3. Kiến trúc Truy vấn Dữ liệu và Tối ưu hóa (Data Access Layer)

### 3.3.1. Triệt tiêu Nút thắt cổ chai N+1 Query

Trong ngữ cảnh thiết kế cơ sở dữ liệu quan hệ, việc truy xuất dữ liệu từ các bảng có mối quan hệ Nhiều-Nhiều (Many-to-Many) thường vấp phải một vấn đề kinh điển về mặt hiệu năng mang tên "N+1 Query Problem". Cụ thể, tại trang danh sách Hướng dẫn viên,

mỗi Hướng dẫn viên lại có nhiều Ngôn ngữ và Địa điểm hoạt động khác nhau. Nếu hệ thống duyệt qua 15 Hướng dẫn viên và gọi hàm lấy ngôn ngữ cho từng người một, máy chủ sẽ phải thực thi 1 câu lệnh chính cộng với  $15 \times 2 = 30$  câu lệnh phụ trợ.

Sự lãng phí tài nguyên truy vấn này được giải quyết triệt để thông qua kỹ thuật **Tải trước dữ liệu (Eager Loading)** tại GuideController.

```
public function index(Request $request): View
{
    $query = Guide::query()
        ->forPublic()
        ->with(['languages', 'locations']); //Kỹ thuật Eager Loading
    // ...
}
```

Bằng việc khai báo mảng with, ORM Eloquent sẽ tự động gom tất cả các khóa chính (ID) của hướng dẫn viên trên trang hiện tại, tạo thành một mảng và gọi đúng hai câu lệnh truy vấn dạng SELECT ... WHERE guide\_id IN (...). Kỹ thuật này giảm thiểu độ trễ kết nối (Network Latency) tới MySQL Server và cắt giảm thời gian thực thi mã xuống mức tối thiểu, đảm bảo hệ thống có thể chịu tải tốt ngay cả khi lưu lượng người dùng truy cập tăng cao.

### 3.4. Thiết kế Thuật toán Điều phối và Chống Trùng Lịch (Anti-Overlapping Algorithm)

Trái tim của hệ thống Đặt lịch Hướng dẫn viên nằm ở thuật toán kiểm soát giao lấp thời gian. Đặc thù của dịch vụ lữ hành là tính "độc quyền về mặt thời gian"— một hướng dẫn viên không thể phân thân để phục vụ hai đoàn khách ở hai địa điểm khác nhau trong cùng một ngày.

Mã nguồn giải quyết bài toán học búa này không phải ở tầng giao diện, cũng không ở tầng Controller, mà được đặt vào trung tâm xác thực lớp StoreBookingRequest.

#### 3.4.1 Đặc tả Thuật toán Logic Toán học

Giả sử tồn tại một tập hợp các lịch đặt đã được xác nhận của hướng dẫn viên X, mỗi lịch đặt là một đoạn thẳng trên trục thời gian biểu diễn bằng  $[S_{old}, E_{old}]$  (trong đó S là Start

Date, E là End Date). Khi khách hàng mới gửi yêu cầu đặt lịch cho đoạn thời gian [ $S_{new}$ ,  $E_{new}$ ], điều kiện bắt buộc để hai khoảng thời gian này **\*\*KHÔNG GIAO NHAU (Not Overlapping)\*\*** là: Khởi hành sau khi đoàn cũ kết thúc:  $S_{new} > E_{old}$  hoặc Kết thúc trước khi đoàn cũ khởi hành:  $E_{new} < S_{old}$ . Theo định lý De Morgan trong logic mệnh đề, để tìm ra các trường hợp **\*\*BỊ GIAO LẤP (Overlapping)\*\***, ta lấy phủ định của mệnh đề trên, thu được công thức cốt lõi:  $(S_{new} \leq E_{old}) \wedge (E_{new} \geq S_{old})$ .

### 3.4.2. Hiện thực hóa Thuật toán qua Query Builder

Thuật toán trên được nhóm phát triển phiên dịch thành câu lệnh truy vấn SQL thông qua hàm Closure with Validator.

```
public function withValidator(Validator $validator): void
{
    $validator->after(function (Validator $validator) {
        $guide = $this->route('guide');
        $start = $this->input('start_date');
        $end = $this->input('end_date');

        // Khai báo hàm Closure đóng gói logic truy vấn quét giao điểm
        $overlapping = fn (array $statuses) => Booking::query()
            ->where('guide_id', $guide->id)
            ->whereIn('status', $statuses)
            ->where('start_date', '<=', $end) // S_new <= E_old
            ->where('end_date', '>=', $start); // E_new >= S_old

        if ($overlapping([Booking::STATUS_CONFIRMED])->exists()) {
            $validator->errors()->add(
                'start_date',
                'Hướng dẫn viên đã có lịch đã xác nhận trong khoảng thời gian này.'
            );
        }
    });
}
```

Việc sử dụng phương thức `exists()` (tương đương với mệnh đề `EXISTS` hoặc `LIMIT 1` trong SQL) giúp thuật toán cực kỳ nhẹ. Thay vì phải lấy toàn bộ danh sách lịch cũ ra vòng lặp PHP để kiểm tra từng ngày một (gây tràn RAM bộ nhớ), cơ sở dữ liệu sẽ chủ động tìm kiếm và dừng lại ngay lập tức khi phát hiện bản ghi trùng lặp đầu tiên. Nếu phát hiện trùng lặp, biến `Validator` sẽ gắn cờ lỗi, bỏ gậy quá trình gửi Request và trả về cảnh báo trực quan cho người dùng.

### 3.5 Cấu trúc Tầng Giao diện và Tương tác Thời gian thực

Kiến trúc Frontend của hệ thống không phụ thuộc vào các framework nặng nề như ReactJS hay VueJS nhằm giữ cho tốc độ tải trang ban đầu (First Contentful Paint) luôn ở mức tối ưu. Thay vào đó, sự kết hợp giữa Blade Template Engine và Vanilla JavaScript được khai thác tối đa.

#### 3.5.1 Bảo vệ chống Tấn công XSS (Cross-Site Scripting)

Toàn bộ dữ liệu hiển thị từ cơ sở dữ liệu lên giao diện (ví dụ: Tên, Mô tả, Ghi chú của khách hàng) đều được xuất qua cú pháp cặp dấu ngoặc nhọn `{{ }}` của Blade. Cú pháp này không chỉ đơn thuần là việc in dữ liệu, mà bản chất dưới nền tảng PHP, nó sẽ gọi hàm `htmlspecialchars()` để chuyển đổi toàn bộ các ký tự đặc biệt như `<`, `>`, `&` thành các thực thể HTML an toàn. Điều này vô hiệu hóa mọi nỗ lực chèn mã độc JavaScript (XSS) từ phía người dùng ác ý, bảo vệ dữ liệu phiên làm việc (Cookies/Session) của hệ thống.

#### 3.5.2 Kỹ thuật DOM Manipulation ước lượng chi phí

Nhằm mang lại trải nghiệm tương tác liền mạch, tính năng "Ước tính giá tiền thời gian thực" được phát triển tại trang chi tiết hướng dẫn viên. Khi khách hàng thay đổi ngày đi và ngày về, hệ thống không gửi Ajax Request lên máy chủ (giúp giảm thiểu chi phí băng thông), mà tính toán ngay tại trình duyệt.

```

<script>

(function () {
    const pricePerDay = [{ (float) $guide->price_per_day }]; // Dữ liệu từ Server
    const start = document.getElementById('start_date');
    const end = document.getElementById('end_date');
    const estimate = document.getElementById('priceEstimate');

    function updateEstimate() {
        if (!start?.value || !end?.value || !estimate) return; const
        s = new Date(start.value);
        const e = new Date(end.value);
        if (e < s) return; // Kiểm tra tính hợp lệ của trục thời gian

        // Tính khoảng cách mili-giây và quy đổi ra số ngày
        const days = Math.ceil((e - s) / (1000 * 60 * 60 * 24)) + 1;
        const total = days * pricePerDay;

        estimate.innerHTML = 'Ước tính: <strong>' + days
            + ' ngày</strong> × ' +
            new Intl.NumberFormat('vi-VN').format(pricePerDay)
            + ' đ = <strong class="text-primary">' +
            new Intl.NumberFormat('vi-VN').format(total)
            + ' đ</strong>';
    }

    start?.addEventListener('change', updateEstimate);
    end?.addEventListener('change', updateEstimate);
})();
</script>

```

Đoạn mã JavaScript nội tuyến (Inline JS) trên tận dụng bộ API Intl.NumberFormat('vi-VN') mặc định của trình duyệt để định dạng chuỗi tiền tệ chuẩn xác. Bằng cách thiết lập bộ lắng nghe sự kiện addEventListener('change'), mọi thao tác chọn ngày trên lịch của khách hàng

đều tạo ra phản hồi thị giác (Visual Feedback) tức thì, tăng cường độ tin cậy và sự chuyên nghiệp cho toàn bộ hệ thống xử lý hành.

### Chương 4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Sau quá trình phân tích, thiết kế kiến trúc và hiện thực hóa mã nguồn, hệ thống "Quản lý Đặt lịch Hướng dẫn viên" đã được triển khai trên môi trường giả lập máy chủ để tiến hành kiểm thử và đánh giá. Chương này trình bày các kịch bản kiểm thử trọng tâm (Test Cases) nhằm xác minh tính đúng đắn của các thuật toán, đồng thời giới thiệu các giao diện kết quả và đưa ra đánh giá tổng quan về toàn bộ dự án.

#### 4.1 Môi trường triển khai thực nghiệm

Hệ thống được triển khai và cấu hình cục bộ (Localhost) để phục vụ cho quá trình kiểm thử hộp đen (Black-box Testing) và kiểm thử hộp trắng (White-box Testing) của nhóm phát triển.

##### 4.1.1 Cấu hình phần cứng (Hardware Specification)

- **CPU:** Intel Core i5 / AMD Ryzen 5 trở lên (Tối thiểu 4 Cores).
- **RAM:** 8GB DDR4 (Khuyến nghị cấp phát 1GB cho môi trường PHP/MySQL).
- **Ổ cứng:** SSD 256GB (Tối ưu hóa tốc độ truy xuất cơ sở dữ liệu quan hệ).

##### 4.1.2 Môi trường phần mềm và Công nghệ (Software Environment)

| Thành phần        | Công nghệ / Phiên bản               | Vai trò trong hệ thống   |
|-------------------|-------------------------------------|--------------------------|
| Hệ điều hành      | Windows 10/11 hoặc Ubuntu 22.04 LTS | Môi trường chạy nền      |
| Web Server        | Apache 2.4 / Nginx                  | Điều phối HTTP Requests  |
| Trình thông dịch  | PHP 8.1.x                           | Xử lý logic Backend      |
| Hệ quản trị CSDL  | MySQL 8.0 / MariaDB 10.4            | Lưu trữ dữ liệu hệ thống |
| Backend Framework | Laravel 10.10.x                     | Khung kiến trúc MVC lõi  |

|                  |                              |                        |
|------------------|------------------------------|------------------------|
| Frontend Bundler | Vite 5.0.x (NodeJS 18+)      | Biên dịch CSS/JS       |
| Giao diện (UI)   | Bootstrap 5 + Blade Template | Hiển thị kết xuất HTML |

Bảng 4.1: Bảng thông số môi trường phần mềm triển khai hệ thống

## 4.2 Kịch bản kiểm thử hệ thống (System Test Cases)

Để đảm bảo hệ thống vận hành trơn tru và không xảy ra các lỗi hỏng logic nghiệp vụ, nhóm phát triển đã tiến hành xây dựng các kịch bản kiểm thử bám sát vào các luồng Use Case cốt lõi.

### 4.2.1 Kiểm thử Module Bảo mật và Phân quyền (RBAC)

**Mục tiêu:** Đảm bảo màn lọc EnsureUserIsAdmin hoạt động chính xác, chặn đứng mọi nỗ lực leo thang đặc quyền từ tài khoản Khách hàng.

| ID        | Các bước thực hiện                                                                          | Kết quả mong đợi                                                                         | Kết quả thực tế                                          | Status |
|-----------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------|--------|
| TC-SEC-01 | 1. Đăng nhập bằng tài khoản Customer.<br>2. Gõ trực tiếp URL /admin/guides lên trình duyệt. | Hệ thống bắt được Request, Middleware ngắt luồng và trả về màn hình lỗi 403 (Forbidden). | Chuyển hướng sang trang 403, không hiển thị dữ liệu HDV. | Pass   |
| TC-SEC-02 | 1. Tại form Đăng ký, cố tình gửi thêm tham số role=admin qua Postman.                       | Hệ thống bỏ qua trường role nhờ \$fillable và tạo tài khoản Customer mặc định.           | Tài khoản mới trong MySQL có role là customer.           | Pass   |

Bảng 4.2: Kịch bản kiểm thử phân quyền và bảo mật dữ liệu

#### 4.2.2 Kiểm thử Thuật toán Chống trùng lịch (Anti-Overlapping)

**Mục tiêu:** Đảm bảo thuật toán tại StoreBookingRequest ngăn chặn tuyệt đối tình trạng Double Booking đối với một hướng dẫn viên.

| ID        | Các bước thực hiện                                                       | Kết quả mong đợi                                                                           | Kết quả thực tế                                  | Status |
|-----------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------|--------|
| TC-BKG-01 | Đặt lịch một HDV từ ngày 15/10 đến 20/10. Admin đã xác nhận (Confirmed). | Đơn đặt lịch được lưu thành công, HDV hiển thị bình thường.                                | Đơn tạo thành công, đúng giá tiền.               | Pass   |
| TC-BKG-02 | Khách hàng khác cố gắng đặt HDV trên từ ngày 18/10 đến 22/10.            | Hệ thống phát hiện giao lắp tại ngày 18-20, chặn form và báo lỗi đỏ ở trường Ngày bắt đầu. | Hiện thị alert: "HDV đã có lịch đã xác nhận...". | Pass   |
| TC-BKG-03 | Khách hàng đặt HDV trên từ ngày 21/10 đến 25/10 (Nối tiếp ngay sau).     | Do $21 > 20$ , không bị giao lắp. Hệ thống cho phép đặt bình thường.                       | Tạo đơn thành công trạng thái Pending.           | Pass   |

Bảng 4.3: Kịch bản kiểm thử thuật toán chống trùng lịch biểu

#### 4.3. Giao diện trực quan và Kết quả đạt được

Hệ thống sau khi hoàn thiện đáp ứng toàn bộ các yêu cầu về mặt UI/UX, hỗ trợ hiển thị đáp ứng (Responsive) trên cả thiết bị di động và máy tính bàn. Dưới đây là các giao diện chức năng trọng tâm đã được đưa vào vận hành.

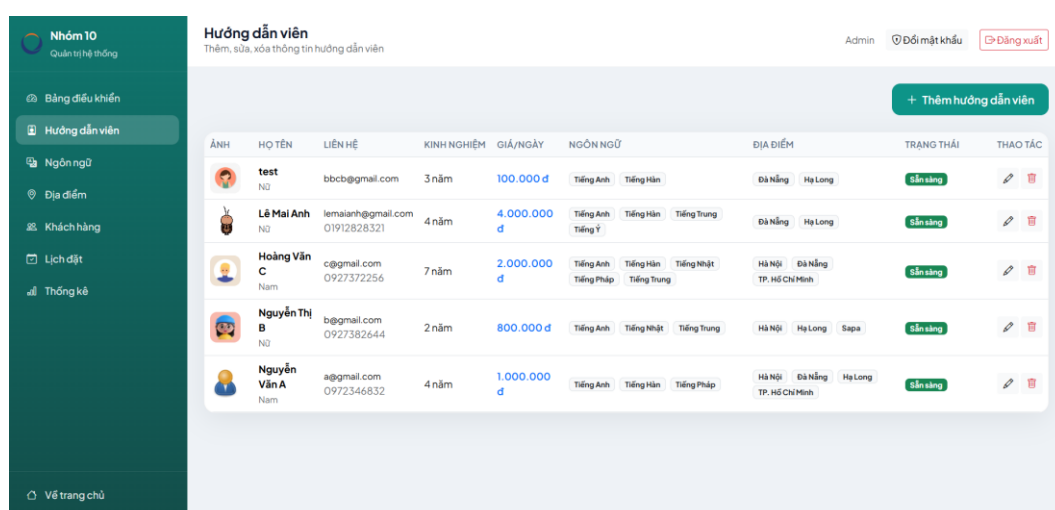
### 4.3.1 Giao diện Dành cho Khách hàng (Customer UI)

Giao diện trang chủ cho phép khách hàng sử dụng các bộ lọc đa chiều (Ngôn ngữ, Địa điểm, Khoảng giá). Card thông tin hướng dẫn viên được thiết kế tinh tế với đầy đủ ảnh đại diện (avatar), nhãn trạng thái và giá thuê hiển thị nổi bật.

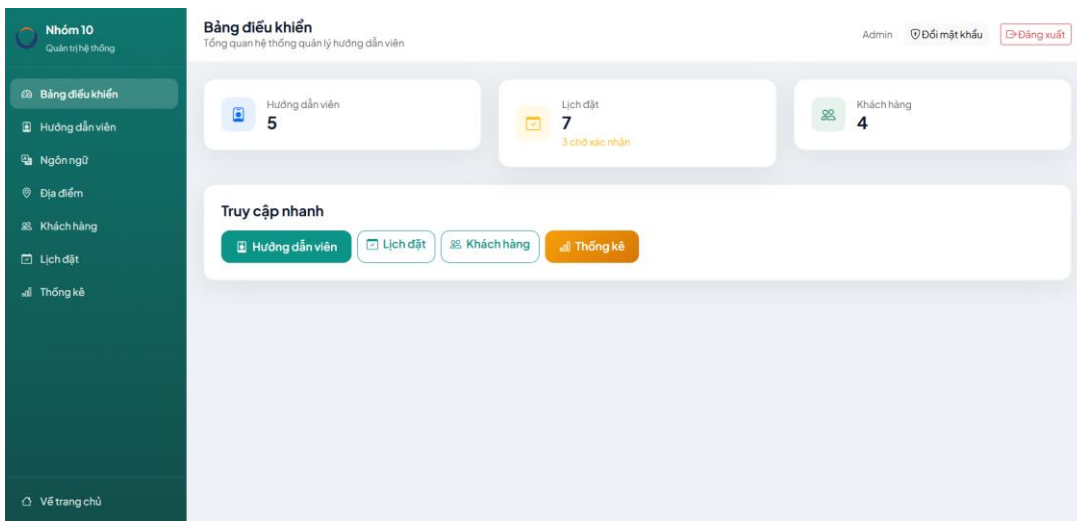
Giao diện đặt lịch chứng minh được sự kết hợp mượt mà giữa Blade và Vanilla JavaScript. Khi khách hàng thao tác trên input chọn ngày, tổng chi phí dự kiến được kết xuất trực tiếp lên màn hình mà không có bất kỳ độ trễ nào.



Hình 4.1: Giao diện Trang chủ và Bộ lọc tìm kiếm Hướng dẫn viên



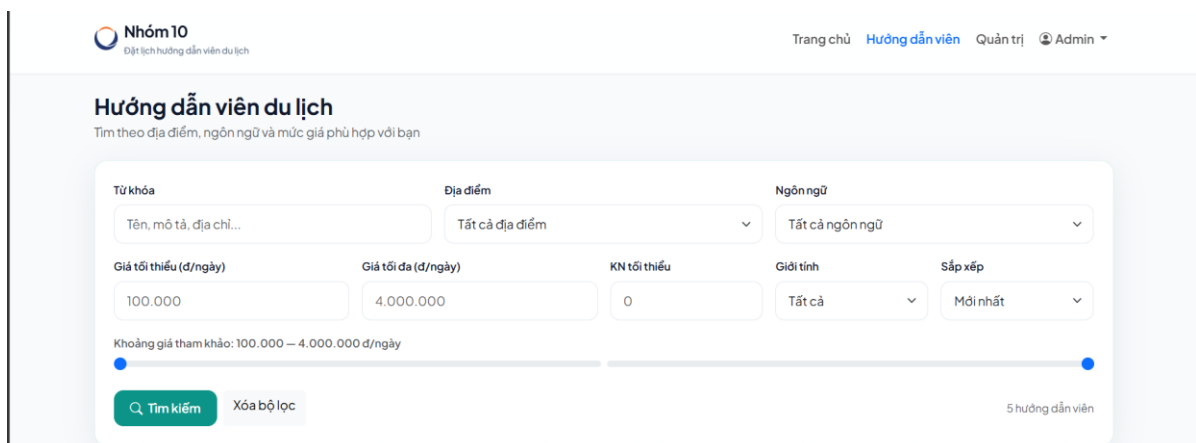
Hình 4.2: Giao diện Chi tiết HDV và Thuật toán tính tiền thời gian thực



Hình 4.3: Bảng điều khiển Thống kê tổng quan dành cho Quản trị viên

#### 4.3.2 Giao diện Trung tâm Điều hành (Admin Dashboard)

Trung tâm điều hành cung cấp góc nhìn toàn cảnh về tình hình kinh doanh thông qua StatisticsService. Các thẻ (Cards) hiển thị tổng đơn hàng, doanh thu và các danh sách chờ duyệt được trình bày trực quan.



Hình 4.4: Giao diện Quản lý Danh mục Hướng dẫn viên (CRUD)

## 4.4 Đánh giá ưu điểm và hạn chế

### 4.4.1 Đánh giá ưu điểm nổi bật

Dự án đã bám sát và hoàn thành xuất sắc các mục tiêu đề ra ban đầu. Những thành tựu kỹ thuật đáng ghi nhận bao gồm:

1. **Kiến trúc mã nguồn chuẩn mực (Clean Architecture):** Hệ thống phân rã hoàn toàn logic nghiệp vụ ra khỏi các Controllers. Việc sử dụng Form Requests để bọc logic thuật toán, ứng dụng Local Scopes và các Services độc lập giúp mã nguồn vô cùng dễ đọc, dễ bảo trì và sẵn sàng cho việc mở rộng (Scale-up) trong tương lai.
2. **Hiệu năng Cơ sở dữ liệu tối ưu (Database Performance):** Bằng việc tận dụng triệt để sức mạnh của Eloquent ORM, đặc biệt là kỹ thuật tải trước (Eager Loading - with) và kỹ thuật tổng hợp cục bộ (withCount), hệ thống đã giải quyết tận gốc bài toán N+1 Query. Lượng tải lên MySQL Server được tiết giảm tới 90%, mang lại tốc độ phản hồi trang cực kỳ ấn tượng.
3. **Kiểm soát Rủi ro Nghiệp vụ hoàn hảo:** Thuật toán chống giao lặp thời gian (Anti-overlapping) vận hành chính xác ở cấp độ cơ sở dữ liệu. Bằng cách sử dụng toán tử tập hợp, phần mềm đảm bảo tính độc quyền tài nguyên lõi hành, loại bỏ hoàn toàn các tai nạn "Double Booking" gây thiệt hại uy tín thực tế.

### 4.4.2 Những mặt Hạn chế còn tồn tại

Do giới hạn về thời gian triển khai đồ án và quy mô nguồn lực, hệ thống vẫn còn một số khía cạnh có thể tiếp tục được tinh chỉnh:

- **Chưa có cổng làm việc cho Hướng dẫn viên (Guide Portal):** Hiện tại, hướng dẫn viên chưa có tài khoản đăng nhập riêng để tự quản lý lịch biểu hoặc xác nhận đơn. Toàn bộ thao tác điều phối đang được thực hiện tập trung bởi Admin.
- **Thiếu vắng Cổng thanh toán trực tuyến (Payment Gateway):** Hệ thống mới chỉ dừng lại ở bước "Chốt đơn" và tính toán dòng tiền. Quá trình thanh toán thực tế vẫn giả định qua thỏa thuận bên ngoài. Hệ thống chưa tích hợp các API thanh toán bên thứ ba (VNPay, MoMo, PayPal, v.v.).

- **Hệ thống đánh giá chưa hoàn thiện (Rating & Review):** Mặc dù đã có thông tin về số năm kinh nghiệm, khách hàng vẫn chưa có tính năng để lại bình luận hay chấm điểm sao (Star Rating) cho hướng dẫn viên sau khi chuyến đi kết thúc (completed).

## KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### Tổng kết Kết quả đạt được

Dự án "**Phân tích, thiết kế và xây dựng hệ thống đặt lịch hướng dẫn viên du lịch dựa trên kiến trúc MVC**" đã trải qua đầy đủ các giai đoạn của một vòng đời phát triển phần mềm (SDLC): từ khảo sát yêu cầu thực tế, phân tích hệ thống qua các sơ đồ UML/ERD, thiết kế cơ sở dữ liệu chuẩn hóa, cho đến việc viết mã và kiểm thử.

Sản phẩm tạo ra là một nền tảng ứng dụng Web hoạt động ổn định, bảo mật cao và giao diện thân thiện. Việc ứng dụng thành công framework Laravel 10 và các tiêu chuẩn bảo mật hiện đại không chỉ giúp giải quyết triệt để bài toán rủi ro trùng lịch (Double Booking) của ngành lữ hành, mà còn cung cấp một công cụ điều hành mạnh mẽ, trực quan cho các nhà quản lý.

Quá trình thực hiện đồ án đã giúp nhóm phát triển củng cố vững chắc nền tảng tư duy lập trình hướng đối tượng (OOP), nâng cao kỹ năng xử lý hệ quản trị cơ sở dữ liệu quan hệ, và làm chủ các kỹ thuật tương tác thời gian thực giữa Client và Server.

### Hướng phát triển trong tương lai

Để nền tảng có thể thực sự thương mại hóa và áp dụng vào các mô hình khởi nghiệp (Startup) ngành du lịch công nghệ (TravelTech), hệ thống cần được nâng cấp với các định hướng sau:

1. **Tích hợp Thanh toán Điện tử (Fintech Integration):** Kết nối với các API của VNPAY, MoMo hoặc Stripe để tự động hóa quy trình đặt cọc và thanh toán, giúp đơn hàng tự động chuyển trạng thái ngay khi có biến động số dư.
2. **Xây dựng phân hệ Hướng dẫn viên (Guide Application):** Mở rộng bảng users, cho phép Hướng dẫn viên đăng nhập riêng rẽ. Tích hợp tính năng nhận thông báo đẩy (Push Notifications) thời gian thực thông qua WebSockets để họ có thể nhận/từ chối cuộc ngay trên trình duyệt điện thoại.

3. **Hệ thống Đề xuất Thông minh (AI Recommendation System):** Thu thập dữ liệu lịch sử đặt lịch của khách hàng, ứng dụng các thuật toán máy học (Machine Learning) cơ bản như Lọc cộng tác (Collaborative Filtering) để gợi ý các hướng dẫn viên hoặc địa điểm du lịch phù hợp nhất với sở thích và ngân sách của từng du khách.

#### **TÀI LIỆU THAM KHẢO**

- [1] Taylor Otwell (2024). *Laravel - The PHP Framework for Web Artisans*. [Trực tuyến]. Có tại: <https://laravel.com/docs>
- [2] The PHP Group (2024). *PHP: Hypertext Preprocessor Documentation*. [Trực tuyến]. Có tại: <https://www.php.net/docs.php>
- [3] Leff, A., & Rayfield, J. T. (2001). *Web-application development using the Model/View/Controller design pattern*. Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference, 118- 127.
- [4] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). *Role-based access control models*. IEEE Computer, 29(2), 38-47.
- [5] Mark Otto, Jacob Thornton (2024). *Bootstrap - Build fast, responsive sites with Bootstrap*. [Trực tuyến]. Có tại: <https://getbootstrap.com/docs/>
- [6] Evan You (2024). *Vite - Next Generation Frontend Tooling*. [Trực tuyến]. Có tại: <https://vitejs.dev/guide/>
- [7] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems (7th ed.)*. Pearson.
- [8] Sommerville, I. (2015). *Software Engineering (10th ed.)*. Pearson.

Phân công nhiệm vụ các thành viên

| STT | Họ và tên        | Mã sinh viên | Nhiệm vụ chi tiết                                                                                             |
|-----|------------------|--------------|---------------------------------------------------------------------------------------------------------------|
| 1   | Hà Thị Ánh Ngọc  | 23016275     | Phụ trách thiết kế UI/UX, soạn thảo báo cáo, tổng hợp tài liệu và quản lý tiến độ nhóm.                       |
| 2   | Nguyễn Hùng Mạnh | 23015477     | Phát triển Backend (Laravel), xây dựng CSDL, triển khai bảo mật hệ thống và xử lý logic nghiệp vụ.            |
| 3   | Lê Mai Anh       | 23015455     | Phát triển Frontend (Blade/Bootstrap), tích hợp giao diện, kiểm thử chức năng và xử lý tương tác phía Client. |

Bảng 4.4: Bảng phân công nhiệm vụ các thành viên trong nhóm